Міністерство освіти і науки України
Національний технічний університет України “КПІ” імені Ігоря Сікорського
Кафедра обчислювальної техніки ФІОТ

ЗВІТ
з Лабораторної роботи №4
з навчальної дисципліни “Технології Computer Vision (сертифікаційна програма Data Science із Sigma Software)”

Тема: **ТЕХНОЛОГІЇ ПОРІВНЯННЯ ЦИФРОВИХ ЗОБРАЖЕНЬ**

Виконав:
Студент 3-го курсу кафедри ОТ ФІОТ
Навчальної групи ІО-43
Шепітко Владислав
Варіант №17

Перевірив:
Професор кафедри ОТ ФІОТ
Олексій Писарчук

Київ-2026

Мета: Дослідити принципи та особливості практичного застосування технологій порівняння цифрових зображень для задач Computer Vision з використанням спеціалізованих програмних бібліотек.
Завдання: За основу взято завдання II-го рівня складності. Група вимог для завдання:

Група вимог №1:

Доповнити результати Лабораторної роботи №2 методами фільтрації та корекції кольору цифрових зображень та розробити / модифікувати програмний скрипт з одним з обраних функціоналів.

1) Обрати сегмент кадру з цікавим для Вас об'єктом, наприклад садиби с. Забір'я, Київська область за даними від https://livingatlas2.arcgis.com/landsatexplorer/ (або інше джерело оперативних даних зі знімками поверхні Землі, наприклад див. джерела даних вище):
2) Обраний в п.1. знайти на ресурсах https://www.bing.com/maps (або інше джерело даних високої роздільної здатності зі знімками поверхні Землі, наприклад див. джерела даних вище):
3) Враховуючи результати цифрової обробки зображень, отримані у ході виконання технічних завдань Лабораторних робіт №2 та №3 підготувати отримані зображення для порівняння обраних об'єктів за джерелами п.1.
4) Оцінити ймовірність ідентифікації як відношення виявлених особливих точок, що належать об'єкту до кількості тих, що придатні та беруть участь в порівнянні.
5) Методами цифрової обробки зображень спробувати отримати ймовірність ідентифікації не гірше 0.9.

Група вимог №2:

Методами та технологіями Object Tracking (виключно без методів глибинного навчання) реалізувати розробку скрипта з обов'язковим функціоналом:

1) Отримання потокового відео з цифрової камери будь-якого типу з присутністю обраного динамічного об'єкту (як виняток дозволяється використовувати стокове відео, наприклад із джерел, вказаних вище);
2) Цифрову обробку потокового відео до рівня, придатного до Object Tracking обраного об'єкту;
3) Реалізацію та дослідження ефективності не менше 3-х відомих методів для Object Tracking обраного об'єкту;
4) Реалізувати аналіз отриманих результатів та їх документування,

Теоретичні відомості:
Для виконання лабораторної роботи було використано теоретичний апарат з минулої роботи (Лабораторна робота №3), та нові теоретичні відомості, специфічні для цієї роботи. Нижче можна детальніше ознайомитись з потрібною, новою теорією для виконання роботи:

Загальні поняття:

- **Особлива точка** - це локальний елемент зображення, виражений певними перепадами яскравості, одразу в декількох напрямках (точка може належати ознаці кута, ребра або площини)

  Головна властивість особливої точки - стійкість до масштабування, зміни ракурсу чи зміні яскравості. Фактично це і є сутністю особливої точки. Саме ці властивості забезпечують можливість детекції об'єктів однієї і тої самої сцени при різних параметрах сцени.

- **Дескриптор особливої точки** - це компактний математичний вектор, сформований в околі цільової особливої точки, який служить особливою ознакою певної особливої точки.

  Головними вимогами до дескриптора є:
  - **Інваріантність** - стійкість векторного представлення до здвигів, поворотів, масштабуванню та фотометричним змінам яскравості.
  - **Унікальність** - дескриптори різних точок повинні максимально відрізнятись одне від одного у векторному просторі ознак.

Математичний апарат кутів Харіса:

Метод кутів Харіса базується на аналізі зміні яскравості кадру при зрушенні невеликого локального вікна у всіх можливих напрямках.

Формування функції зовнішнього зрушення:

$$E(u,v) = \sum_{x,y} w(x,y) \cdot [I(u + x, v + y) - I(x,y)]^2$$

де:

- $E(u,v)$ - функція зміни інтенсивності. Вона показує наскільки сильно змінюється картинка при її зрушенні на вектор (u,v).
- $w(x,y)$ - функція вікна:
  - Прямокутне вікно: $w(x,y) = 1$ всередині вікна та 0 - ззовні.

- Гаусеве вікно: $w(x,y) = \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$, забезпечує кругову ізотропність (тобто симетричний розподіл по колу, в залежності від відстані цільової точки, до центру гаусового вікна) та згладжування
- $I(x+u, y+v)$ - зміщенна яскравість кадра, $I(x,y)$ - поточна яскравість.

Використовуючи перетворення функції у ряд Тейлора першого порядку, маємо настпуний вигляд для функції інтенсивності світла (яскравості):

$$I(u+x, v+y) \approx I(x,y) + u \cdot I_x(x,y) + v \cdot I_y(x,y)$$

Тоді підставивши це представлення функції яскравості світла у початкову формулу, отримаємо наступний вигляд квадрату різниці зміщення:

$$E(u,v) = \sum_{x,y} w(x,y) \cdot (u^2 I_x^2 + 2uvI_x I_y + v^2 I_y^2)$$

Бачимо, що формула набуває квадратичного вигляду. Можемо представити цю квадратичну формулу у матричному вигляді, використавши “трюк” лінійної алгебри. Тоді матимемо наступний запис:

$$E(u,v) \approx \begin{bmatrix} u & v \end{bmatrix} \cdot \mathbf{M} \cdot \begin{bmatrix} u \\ v \end{bmatrix}$$

де:

$$\mathbf{M} = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \begin{bmatrix} \sum w I_x^2 & \sum w I_x I_y \\ \sum w I_x I_y & \sum w I_y^2 \end{bmatrix} = \begin{bmatrix} M_{xx} & M_{xy} \\ M_{xy} & M_{yy} \end{bmatrix}$$

Саме матриця M є основою для розуміння алгоритмом чи є певна область кутом, ребром чи гранню.

Для цього використовується функція відгуку Харріса:

$$R = \det(M) - k \cdot (trace(M))^2 = (M_{xx} M_{yy} - M_{xy}^2) - k \cdot (M_{xx} + M_{yy})^2$$

де k - емпірична константа, яка належить діапазону $k \in [0.04, 0.06]$.

Тоді, на основі R можна робити наступні висновки:
$R > 0$: Область є кутом
$R < 0$: Область є ребром
$|R| \approx 0$: Область є плоскою поверхнею

Метод кутів Харріса є гарним варіантом для використання при незмінному масштабі, так як він є інваріантним до яскравості та куту повороту.

Далі, для порівняння особливих кутів, використовуються дескриптори. Щоб порівняти знайдений кут P1 на кадрі A з кутом P2 на кадрі B, навколо них вирізаються прямокутні блоки яскравості $I_1(x, y)$ та $I_2(x, y)$ розміру W x W

Нормована взаємна кореляція (Normalized Cross-Correlation - NCC):

$$c(I_1, I_2) = \frac{\sum_{x,y}(I_1(x,y)-\bar{I}_1)(I_2(x,y)-\bar{I}_2)}{\sqrt{\sum_{x,y}(I_1(x,y)-\bar{I}_1)^2 \cdot \sum_{x,y}(I_2(x,y)-\bar{I}_2)^2}}$$

Значення $c \in [-1, 1]$. Чим ближче до 1.0, тим зібг є точнішим

Сума квадратних різниць (Sum of Squared Differences - SSD):

$$SSD(I_1, I_2) = \sum_{x,y} \left[I_1(x, y) - I_2(x, y)\right]^2$$

Сума абсолютних різниць (Sum of Absolute Differences - SAD):

$$SAD(I_1, I_2) = \sum_{x,y} \left|I_1(x, y) - I_2(x, y)\right|$$

Результати виконання лабораторної роботи:
В ході виконання лабораторної роботи було модифіковано код детекції будівель з минулої лабораторної роботи, для групи вимог №1, з додаванням інваріантного алгоритму детекції особливих точок SIFT, для порівняння детекції будівель з фото району КПІ на мапах Google Maps та Bing Maps. Для групи вимог №2 було створено інший файл **object_tracker.py**

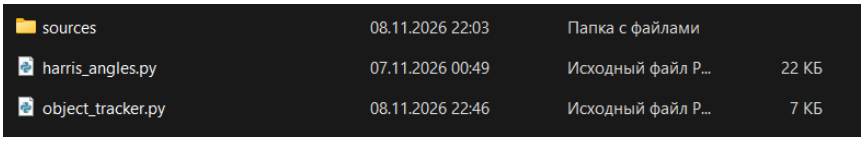


Рис. 1. Структура проекту Лабораторної роботи №4

- **sources** - директорія з супутниковими зображеннями для детекції та відеофайлами для трекінгу об’єкту.
- harris_angles.py - файл для порівняння детекцій з обох супутникових зображень.
- object_tracker.py - файл для реалізації трекінгу об’єктів з використанням трбох алгоритмів на вибрі (KCF, CSRT, MOSSE)

Скріншоти роботи програми:

Для групи вимог №1:

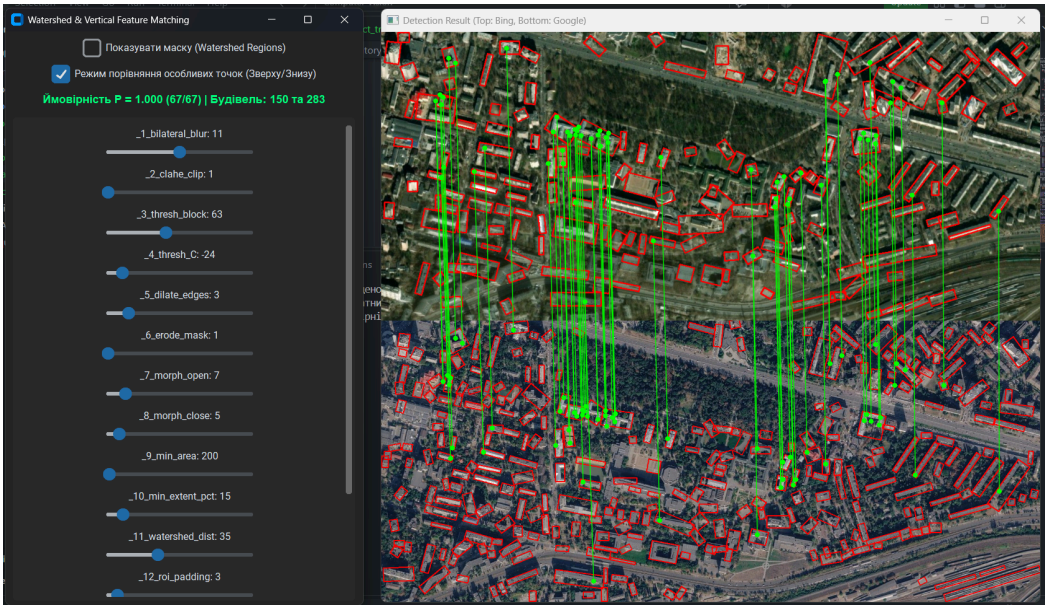


Рис. 2. Порівняння детекції будівель з використанням алгоритму SIFT

Детальніший аналіз результатів:

- Знайдено будівель (Зверху Bing / Знизу Google): 150 / 283
- Придатних точок у порівнянні: 67 | Належать об'єктам: 67
- Ймовірність ідентифікації P: 1.0000 (Ціль >= 0.90)

Також початковий пайплайн (головний алгоритм) було модифіковано для кращої детекції будівель на супутникових знімках, при ортофотоплановій зйомці (якість детекції на знімку Google Maps збільшилась на ~5% при незмінних початкових параметрах), використовуючи для цього операцію морфологічного градієнту (різниця дилатації та ерозізії). Фактично це дозволило штучно створити ознаку тих самих “білих стін” як на Bing Maps, що і покращило детекцію будівель.

Для групи вимог №2:

За вимогами даної групи, потрібно використати три алгоритми об’єктного трекінгу, проаналізувати результати і порівняти. Я взяв наступні алгоритми:

- CSRT (Channel and Spatial Reliability Tracker)
- KCF (Kernelized Correlation Filters)
- MOSSE (Minimum Output Sum of Squared Error)

Нижче, приведено скріншоти з роботою програми детекції та фінальними аналізами алгоритмів:

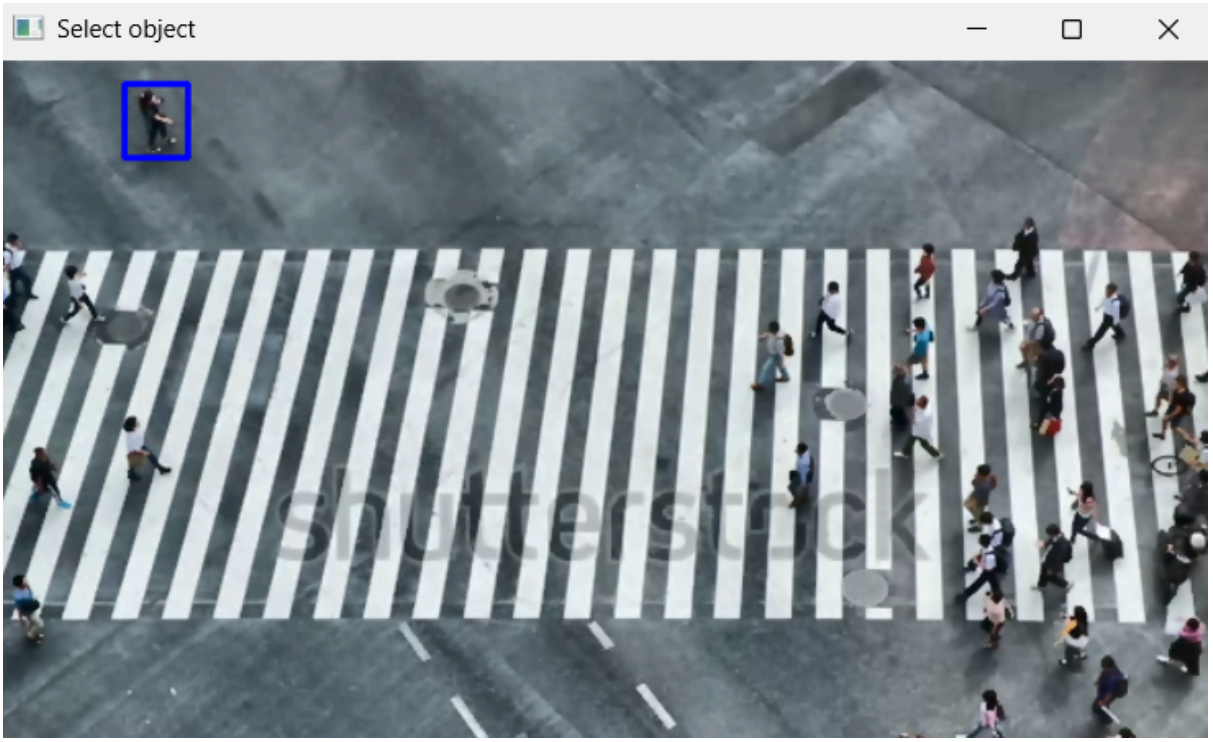


Рис. 3. Вибір об’єкту детекції з першого кадру відео

Для об’єктного трекінгу було використано відео з YouTube для тренування очей, щоб легше було порівняти результати трекінгу об’єктів при “ідеальних” умовах з різними алгоритмами детекції (це було зроблено через те, що на деяких початкових відео, запропонованих методичкою до виконання лабораторної роботи, алгоритми більшу частину відеопотоку перебували у стані “встрачено”, що не дозволяло нормально порівняти алгоритми за застосуванням обчислювальних ресурсів. Проте початкові відео були використані для загального аналізу недоліків вибраних алгоритмів)

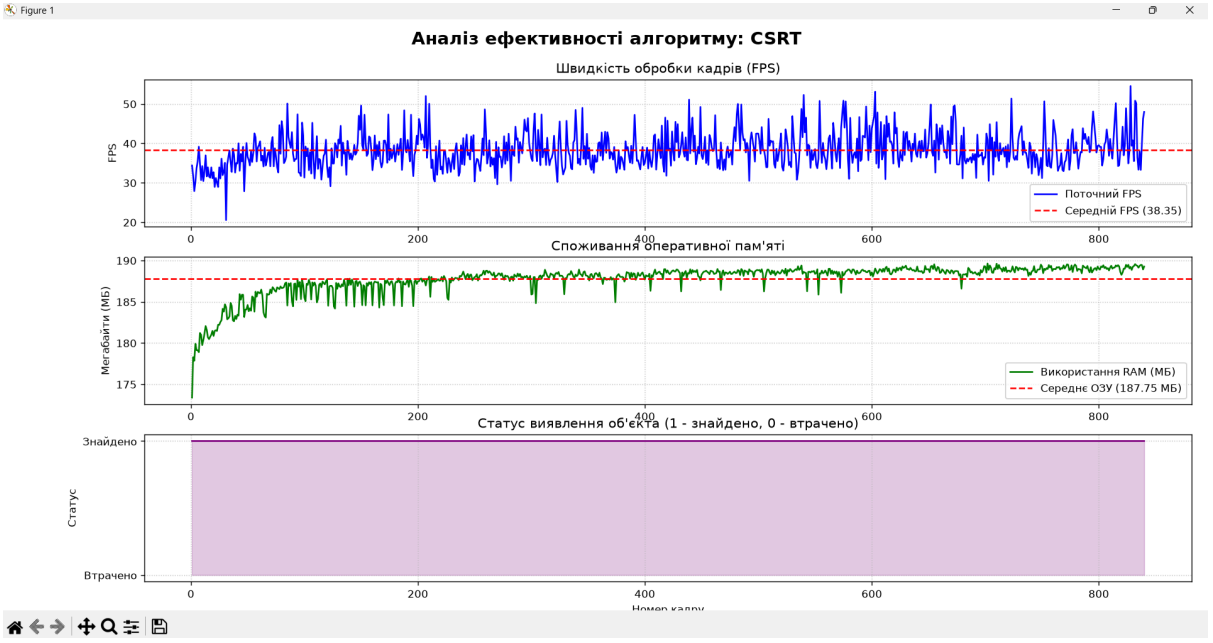


Рис. 4. Аналіз алгоритму CSRT

==========

Детальний аналіз: Алгоритм CSRT

==========

Всього оброблено кадрів: 840

Середня кількість кадрів за секунду (FPS): 38.35 кадрів/с

Точність відстеження об'єкту: 100.00%

Середнє використання ОП: 187.75 МБ

Пікове використання ОП: 189.63 МБ

==========

Як бачимо, алгоритм продемонстрував гарну роботу трекінгу об’єкту, не втрачаючи об’єкт при його переміщенні, проте й використав достатню кількість обчислювальних ресурсів, що призвело до падіння FPS.

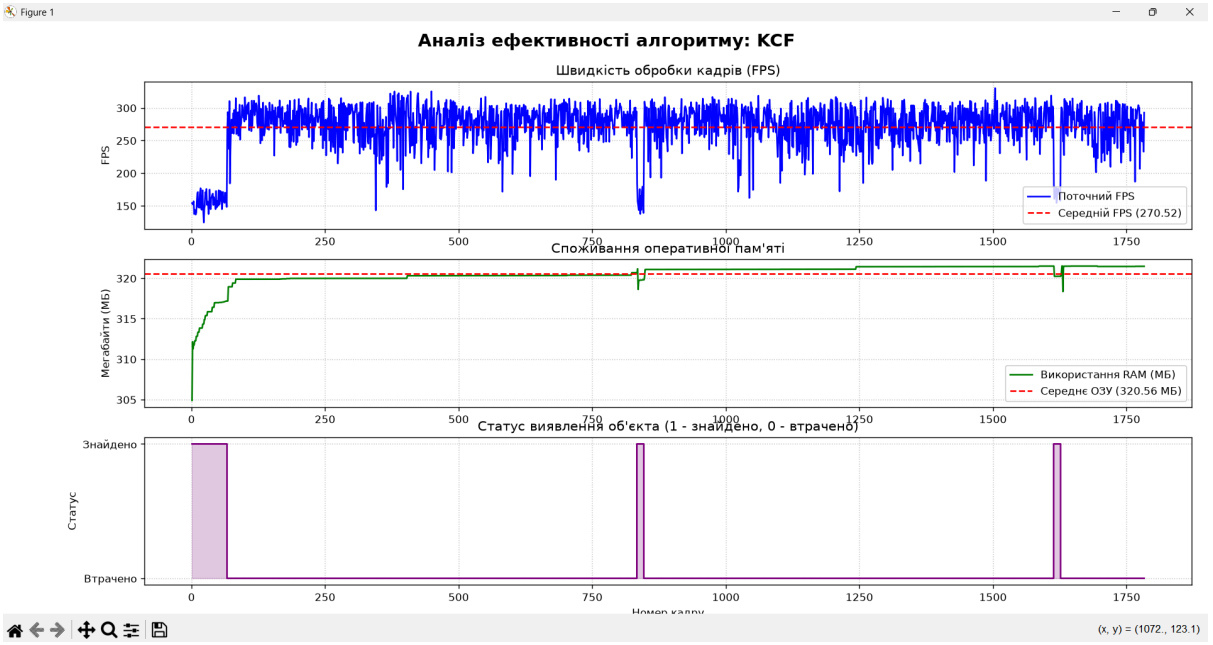


Рис. 5. Аналіз алгоритму KCF

==========

Детальний аналіз: Алгоритм KCF

==========

Всього оброблено кадрів: 1783

Середня кількість кадрів за секунду (FPS): 270.52 кадрів/с

Точність відстеження об'єкту: 5.16%

Середнє використання ОП: 320.56 МБ

Пікове використання ОП: 321.51 МБ

==========

Відеопотік для цього алгоритму спеціально був запущений на більш довгий час, щоб наочно продемонструвати проблему вибраного алгоритму, навіть на досить примітивному для трекінгу відео.

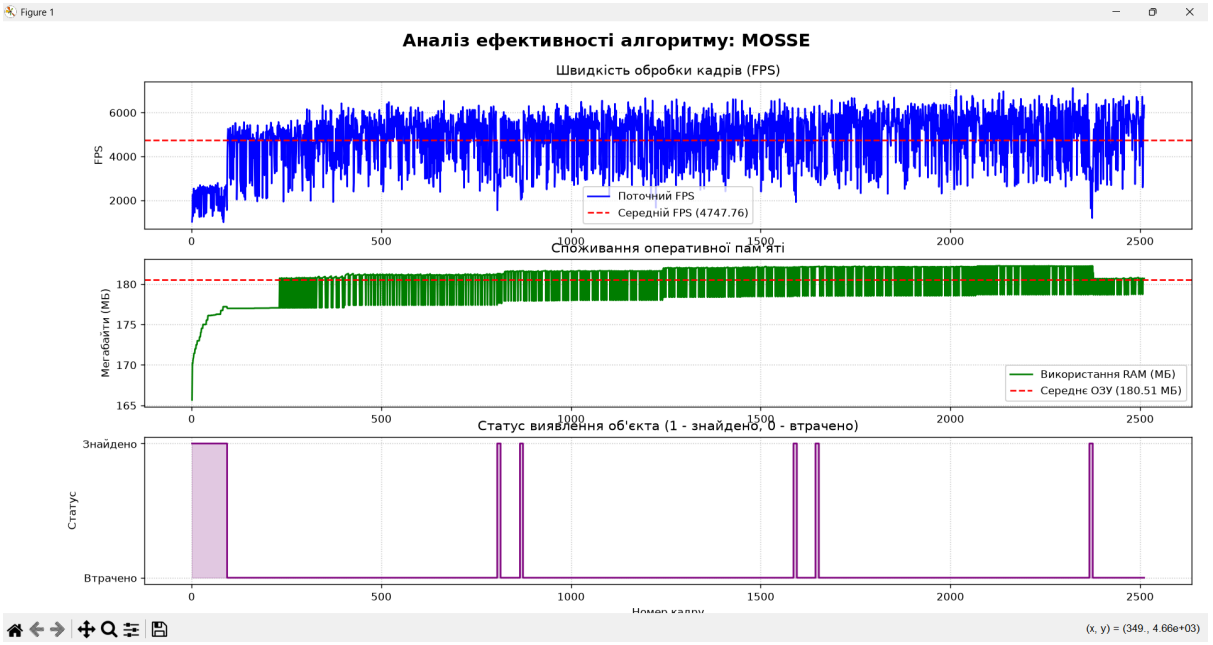


Рис. 6. Аналіз алгоритму MOSSE

==========
Детальний аналіз: Алгоритм MOSSE
==========
Всього оброблено кадрів: 2511
Середня кількість кадрів за секунду (FPS): 4747.76 кадрів/с
Точність відстеження об'єкту: 5.46%
Середнє використання ОП: 180.51 МБ
Пікове використання ОП: 182.26 МБ
==========

Як бачимо, початково здавалось, що відеоряд є досить примітивним для трекінгу, проте алгоритми KCF та MOSSE впорались досить погано з ним. З чим це пов’язано?

Справа в тому, що алгоритми MOSSE та KCF працюють за принципом кореляційних фільтрів. Вони шукають не геометричну форму об'єкта, а його текстуру - унікальний візерунок із дрібних деталей, перепадів світла та контрастних ліній. У нашому випадку чорна крапка є абсолютно суцільною та однотонною. Для математики цих алгоритмів вона виглядає як порожня пляма, всередині якої немає за що зачепитися. Через це алгоритми починають шукати деталі на краях і знаходять їх у кутах квадратної рамки, якою ми початково виділили нашу ціль.

У ці кути неминуче потрапляє шматок зелено-жовтого фону. Оскільки сама крапка не має текстури, трекер запам'ятовує саме ці зелені шматочки фону як головну особливість нашого об'єкта.

Спробуємо знову запустити алгоритми KCF та MOSSE на іншому відеоряді (для тестування використаємо відео **V_7.mp4**):

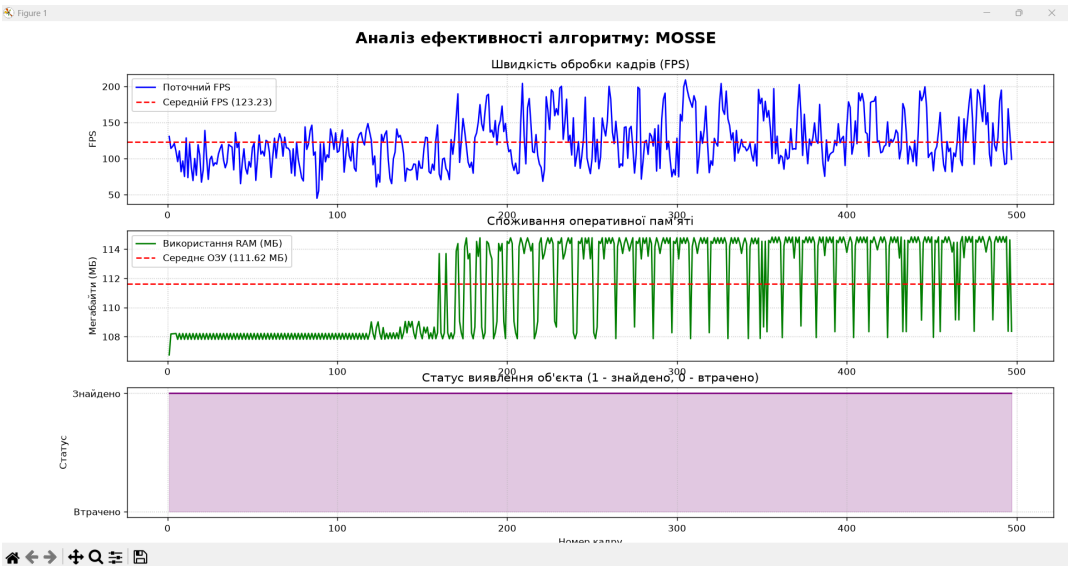


Рис. 7. Аналіз алгоритму MOSSE на іншому відео

==========

Детальний аналіз: Алгоритм MOSSE

==========

Всього оброблено кадрів: 497

Середня кількість кадрів за секунду (FPS): 123.23 кадрів/с

Точність відстеження об'єкту: 100.00%

Середнє використання ОП: 111.62 МБ

Пікове використання ОП: 114.86 МБ

==========

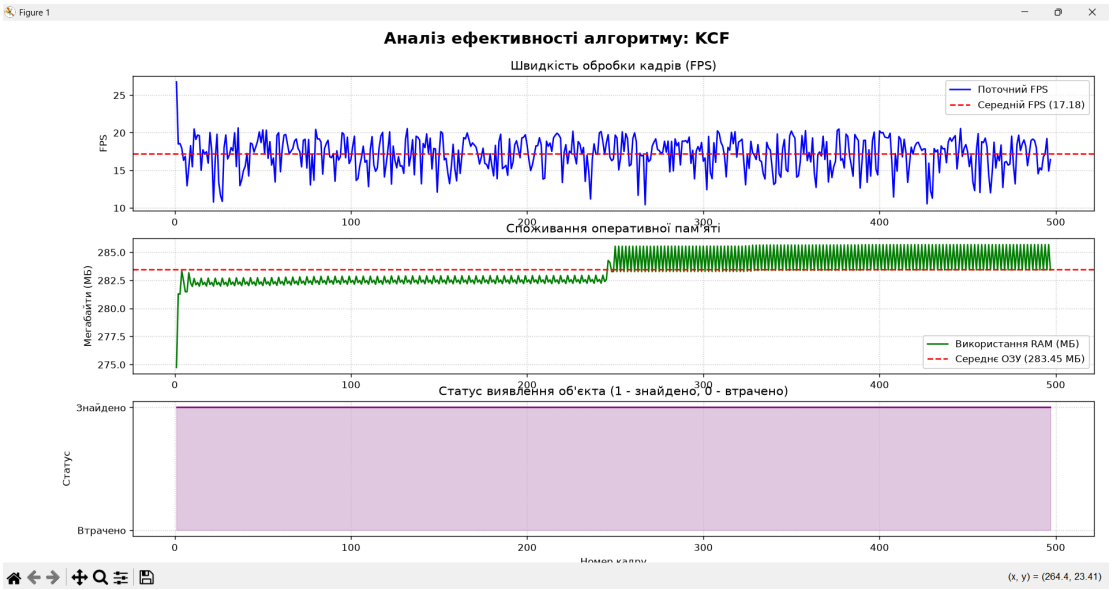


Рис. 8. Аналіз алгоритму KCF на іншому відео

==========
Детальний аналіз: Алгоритм KCF
==========
Всього оброблено кадрів: 497
Середня кількість кадрів за секунду (FPS): 17.18 кадрів/с
Точність відстеження об'єкту: 100.00%
Середнє використання ОП: 283.45 МБ
Пікове використання ОП: 285.68 МБ
==========

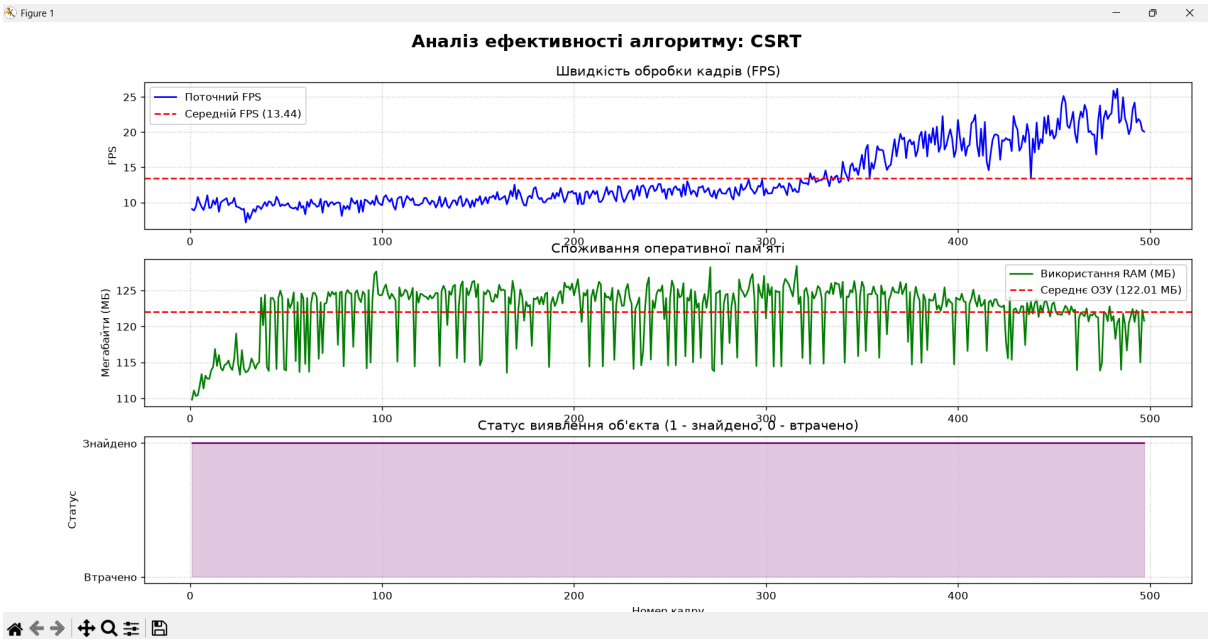


Рис. 9. Аналіз алгоритму CSRT на іншому відео

==========
Детальний аналіз: Алгоритм CSRT
==========
Всього оброблено кадрів: 497
Середня кількість кадрів за секунду (FPS): 13.44 кадрів/с
Точність відстеження об'єкту: 100.00%
Середнє використання ОП: 122.01 МБ
Пікове використання ОП: 128.41 МБ
==========

На цьому відеофрагменті, обидва алгоритми MOSSE та KCF відпрацювали відносно непогано, проте KCF все ж таки відпрацював по якості навіть краще за CSRT (форма рамки майже не деформувалась і залишалась +- як і на початку, в порівнянні з MOSSE). Так як рамка була більша і сам фон був більш текстурний (тобто мав більше унікальних ознак) за який би можна було чіплятись, то й алгоритми спрацювали набагато краще.

Висновок: Отже, під час виконання лабораторної роботи було досліджено методи виявлення та співставлення об’єктів на зображеннях в умовах зміни яскравості та кута повороту, шляхом використання кутів Харріса для зображень без зміни масштабу та інших алгоритмів для створення дескрипторів особливих точок, які є інваріантними до зміни масштабу. Також було практично порівняно переваги та недоліки трьох алгоритмів захоплення об’єктів на відеопотоці, а саме алгоритми MOSSE, KCF та CSRT. Використання зазначених алгоритмів має різну цінність при певних умовах використання. Для стабільних об'єктів із багатою текстурою та достатнім розміром рамки доцільно застосовувати високошвидкісні кореляційні алгоритми (KCF чи MOSSE) для економії ресурсів, тоді як для безтекстурних об'єктів або сцен із перекриттями необхідний обчислювально важкий, але просторово надійний алгоритм CSRT.